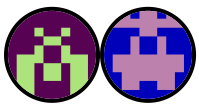
Учебник

Используйте Ansible Vault для защиты конфиденциальных данных плейбуков

Обновлено 29 июня 2026 года

Безопасность Убунту Ансибль Управление конфигурацией



Авторы: Джастин Эллингвуд и Маникандан Куруп

Содержание

Популярные темы

Введение

Ansible Vault — это функция, которая позволяет шифровать значения и структуры данных в проектах Ansible. Это дает возможность защитить любые конфиденциальные данные, необходимые для успешного выполнения задач Ansible, но не предназначенные для публичного просмотра, например пароли, ключи API или закрытые ключи. Ansible автоматически расшифровывает содержимое, зашифрованное с помощью Vault, во время выполнения при вводе правильного пароля.

Из этого руководства вы узнаете, как работает Ansible Vault, как шифровать целые файлы и отдельные переменные, как вводить пароль хранилища в Ansible интерактивно или через файл паролей, а также как управлять несколькими паролями хранилища с помощью идентификаторов хранилища. Вы также узнаете, чем Ansible Vault отличается от других подходов к управлению секретными данными. Вы будете использовать одну управляющую машину Ansible, удаленные хосты не требуются. Это руководство было протестировано на сервере Ubuntu, но рабочий процесс идентичен для любого дистрибутива Linux, на котором установлен Ansible.

Ключевые рекомендации на вынос

- Ansible Vault шифрует неактивные секреты, поэтому вы можете хранить конфиденциальные и неконфиденциальные данные конфигурации в одном проекте, не раскрывая секреты.
- Вы можете зашифровать весь файл с помощью `ansible-vault encrypt` или зашифровать одно значение с помощью `ansible-vault encrypt_string`, что удобнее для общих репозиториев.
- Команды `ansible` и `ansible-playbook` автоматически расшифровывают содержимое хранилища, если указан пароль через `--ask-vault-pass`, `--vault-password-file`, или переменную окружения `ANSIBLE_VAULT_PASSWORD_FILE`.
- Файл с паролем к хранилищу не должен находиться в системе контроля версий (добавьте его в `.gitignore`) и должен быть защищен с помощью `chmod 600`.
- Идентификаторы Vault (синтаксис `--vault-id label@source`) позволяют управлять несколькими паролями, например отдельными секретами для разработки, промежуточной и рабочей среды.
- В конвейерах CI/CD сохраняйте пароль Vault как защищенный секрет конвейера, записывайте его во временный файл во время выполнения и удаляйте этот файл после завершения работы.
- Ansible Vault защищает данные только на диске. После расшифровки во время выполнения секреты хранятся в памяти и могут отображаться в подробном выводе или журналах, поэтому обращайтесь с ними осторожно.
- Если вы потеряете пароль от хранилища, зашифрованные данные будет невозможно восстановить, поэтому важно хранить пароли от хранилищ в специальном менеджере паролей или секретов.
- Для больших команд, расширяющих возможности автоматизации, подойдет внешний менеджер секретов, например HashiCorp Vault может дополнить или заменить Ansible Vault.

Предварительные условия

Чтобы следовать инструкциям, вам понадобится сервер Ubuntu с пользователем, не являющимся **root**, и `sudo` привилегиями. Вы можете создать пользователя с

соответствующими разрешениями, следуя нашему руководству «Начальная настройка сервера с Ubuntu».

На сервере вам нужно будет установить и настроить Ansible. Вы можете воспользоваться нашим руководством по установке Ansible в Ubuntu, чтобы установить соответствующие пакеты. Продолжайте следовать этому руководству, когда ваш сервер будет настроен в соответствии с указанными выше требованиями.

Что такое Ansible Vault?

Ansible Vault is a mechanism that allows encrypted content to be incorporated transparently into Ansible workflows. A utility called `ansible-vault` secures confidential data, called *secrets*, by encrypting it on disk. To integrate these secrets with regular Ansible data, both the `ansible` and `ansible-playbook` commands, used for executing ad hoc tasks and structured playbooks respectively, support decrypting vault-encrypted content at runtime.

Vault is implemented with file-level granularity, meaning that individual files are either encrypted or unencrypted as a whole. It can also encrypt single values inline, which you will see later. Ansible identifies and decrypts any vault-encrypted content it finds while executing a playbook or task, as long as it has the right password.

Encryption algorithm and security model

Ansible Vault encrypts content with AES-256 in CTR mode and derives the encryption key from your password using PBKDF2 with HMAC-SHA256, run over 10,000 iterations with a 32-byte random salt, then authenticates the ciphertext with an HMAC-SHA256 digest. These are strong, industry-standard primitives, so in practice the security of a vault comes down to the strength of the password you choose and how carefully you store it. A short or reused password undermines otherwise solid cryptography.

It is equally important to understand what Ansible Vault does *not* protect against. Vault secures data only while it sits on disk. Once Ansible decrypts a secret to use it during a run, that value exists in memory as an ordinary variable and can appear in verbose output (for example, when you run with `-vvv`), in task results, or in logs if you are not careful. Vault also has no per-user access control beyond the shared password: anyone with the password can read everything that password protects. Treat it as encryption at rest, not as a complete secrets-management platform.

Now that you understand what Vault is and how it secures data, you can start using the tools Ansible provides.

Setting the Ansible Vault editor

Before using the `ansible-vault` command, it is a good idea to specify your preferred text editor. Several of Vault's commands open an editor to manipulate the contents of an encrypted file. Ansible reads the `EDITOR` environment variable to find your preferred editor, and if it is unset, it defaults to `vi` .

If you do not want to edit with the `vi` editor, set the `EDITOR` variable in your environment.

> **Note:** If you find yourself within a `vi` session accidentally, you can exit by pressing the **Esc** key, typing `:q!` , and then pressing **Enter**. If you are not familiar with `vi` , any changes you make are likely to be unintentional, so this command exits without saving.

To set the editor for a single command, prepend the command with the environment variable assignment, like this:

Copy

```
$ EDITOR= nano  ansible-vault  . . .
```

To make this persistent, open your `~/.bashrc` file:

Copy

```
$ nano ~/.bashrc
```

Specify your preferred editor by adding an `EDITOR` assignment to the end of the file:

~/.bashrc

```
export EDITOR= nano
```

Save and close the file when you are finished, then source the file again to read the change into the current session:

Copy

```
$ . ~/.bashrc
```

Display the `EDITOR` variable to check that your setting was applied:

Copy

```
$ echo $EDITOR
```

The command prints the editor you set:

Output Copy

```
nano
```

Now that you have established your preferred editor, you can explore the operations available with the `ansible-vault` command.

Managing sensitive files with `ansible-vault`

The `ansible-vault` command is the main interface for managing encrypted content within Ansible. You use it to initially encrypt files and then to view, edit, rekey, or decrypt the data. The following subsections walk through each operation.

Creating new encrypted files

To create a new file encrypted with Vault, use the `ansible-vault create` command and pass in the name of the file you wish to create. For example, to create an encrypted YAML file called `vault.yml` to store sensitive variables, type the following:

Copy

```
$ ansible-vault create  vault.yml
```

You are prompted to enter and confirm a password:

Output Copy

```
New Vault password:
Confirm New Vault password:
```

When you have confirmed your password, Ansible immediately opens an editing window where you can enter your desired contents. To test the encryption, enter some sample text:

vault.yml Copy

```
Secret information
```

Ansible encrypts the contents when you close the file. If you check the file, instead of the words you typed, you see an encrypted block:

Copy

```
$ cat  vault.yml
```

The file now contains a Vault header followed by the encrypted payload:

Output

```
$ANSIBLE_VAULT;1.1;AES256
65316332393532313030636134643235316439336133363531303838376235376635373430336
396335363037316135663837636164633835376336343436 0a3631383761636662653364336333
303362333236643064346263636437316265366438333663835666139636431366636623161 6
376465365313263620a383666383233626665376364323062393462373266663066366536306
316437313436663537616335636336343261393962303137343330346532383031 66
```

The header tells Ansible how to handle the file, and it is followed by the encrypted contents, which display as hexadecimal digits.

Encrypting existing files

If you already have a file that you wish to encrypt with Vault, use the `ansible-vault encrypt` command instead. For testing, create an example file:

Copy

```
$ echo 'unencrypted stuff' > encrypt_me.txt
```

Now encrypt the existing file:

Copy

```
$ ansible-vault encrypt encrypt_me.txt
```

Again, you are prompted to provide and confirm a password, after which a message confirms the encryption:

Output Copy

```
New Vault password:
Confirm New Vault password:
Encryption successful
```

Instead of opening an editing window, `ansible-vault` encrypts the contents of the file and writes it back to disk, replacing the unencrypted version. If you check the file, you

see a similar encrypted pattern:

Copy

```
$ cat  encrypt_me.txt
```

The output shows the same Vault header and encrypted body as a newly created file:

Output

```
$ANSIBLE_VAULT;1.1;AES256
666339366538346161303464363538653036653964303834303533666162633231613936393937373165393534346664383730356531323834343033386640a396635313062386464306132313343133363133386235373333323562313864386665656165376165386534653334313066386433636663633363562320a6136613139663763613963363838646566323761343530396636626663939363934396636356563616131633964303339313262663930333237333937666 4
```

As you can see, Ansible encrypts existing content in much the same way as it encrypts new files.

Viewing encrypted files

Sometimes you need to reference the contents of a vault-encrypted file without editing it or writing it to the filesystem unencrypted. The `ansible-vault view` command feeds the contents of a file to standard output, which by default displays them in the terminal. Pass the vault-encrypted file to the command:

Copy

```
$ ansible-vault view  vault.yml
```

You are asked for the file's password, and after entering it successfully the contents are displayed:

Output

Copy

```
Vault password:
Secret information
```

The password prompt is mixed into the output of the file contents, so keep this in mind when using `ansible-vault view` in automated processes.

Editing encrypted files

When you need to edit an encrypted file, use the `ansible-vault edit` command:

Copy

```
$ ansible-vault edit  vault.yml
```

You are prompted for the file's password, and after entering it, Ansible opens the file in an editing window where you can make any necessary changes. Upon saving, the new contents are encrypted again with the file's password and written to disk.

Manually decrypting encrypted files

To decrypt a vault-encrypted file, use the `ansible-vault decrypt` command.

> **Note:** Because of the increased likelihood of accidentally committing sensitive data to your project repository, the `ansible-vault decrypt` command is best reserved for when you want to remove encryption from a file permanently. If you only need to view or edit a vault-encrypted file, use `ansible-vault view` or `ansible-vault edit` instead.

Pass in the name of the encrypted file:

Copy

```
$ ansible-vault decrypt  vault.yml
```

You are prompted for the encryption password, and once you enter the correct one, the file is decrypted:

Output Copy

```
Vault password:
Decryption successful
```

If you view the file again, you see the actual contents instead of the encrypted block:

Copy

```
$ cat  vault.yml
```

The file is now plain text on disk:

Output Copy

```
Secret information
```

Your file is now unencrypted on disk, so be sure to remove any sensitive information or re-encrypt the file when you are finished.

Changing the password of encrypted files

If you need to change the password of an encrypted file, use the `ansible-vault rekey` command:

Copy

```
$ ansible-vault rekey  encrypt_me.txt
```

When you enter the command, you are first prompted for the file's current password:

Output Copy

```
Vault password:
```

After entering it, you are asked to select and confirm a new vault password:

Output Copy

```
Vault password:
 New Vault password:
 Confirm New Vault password:
```

When you have successfully confirmed a new password, you receive a message indicating the re-encryption succeeded:

Output Copy

```
Rekey successful
```

The file is now accessible using the new password, and the old password no longer works.

Encrypting individual variables with `ansible-vault encrypt_string`

Encrypting an entire file is useful, but it has a drawback for shared projects: a reviewer cannot see which variables a file defines without decrypting it. The `ansible-vault encrypt_string` command solves this by encrypting a single value inline, so you can keep one readable variables file where most values are plain text and only the sensitive ones are encrypted.

To encrypt a value and assign it a variable name, run the command and pass the value along with the `--name` flag:

Copy

```
$ ansible-vault encrypt_string ' supersecretpassword ' --name ' mysql_passwor
```

You are prompted for a vault password, and then Ansible prints a ready-to-paste YAML snippet:

Output Copy

```
New Vault password:
Confirm New Vault password:
mysql_password: !vault |
          $ANSIBLE_VAULT;1.1;AES256
          396134633437663538613932363632323838313566363730333862306232613065
          6566383437333436353331353764633033366563343631380a316234383...
Encryption successful
```

You can paste that block directly into a regular variables file, mixing encrypted and unencrypted values in one place:

vars.yml Copy

```
---
# nonsensitive data
mysql_port: 3306
mysql_host: 10.0.0.3
mysql_user: fred

# sensitive data, encrypted inline
mysql_password: !vault |
          $ANSIBLE_VAULT;1.1;AES256
          396134633437663538613932363632323838313566363730333862306232613065
          6566383437333436353331353764633033366563343631380a316234383...
```

The `!vault` tag tells Ansible that the value is encrypted and should be decrypted at runtime. This approach keeps variable names visible in code review while protecting only the secret values, which is why it is often preferred over encrypting whole files in collaborative repositories.

Running Ansible with vault-encrypted files

After you encrypt your sensitive information with Vault, you can use the files with Ansible's conventional tooling. The `ansible` and `ansible-playbook` commands both know how to decrypt vault-protected content given the correct password, and there are a few different ways to provide that password depending on your needs.

To follow along, you need a vault-encrypted file. Create one with the following:

Copy

```
$ ansible-vault create secret_key
```

Select and confirm a password, then fill in whatever dummy contents you want:

secret_key

```
confidential data
```

Save and close the file. Next, create a temporary `hosts` file to act as an inventory:

Copy

```
$ nano hosts
```

Add just the Ansible localhost to it. To prepare for a later step, place it in the `[database]` group:

hosts

```
[database]
localhost ansible_connection=local
```

Save and close the file. Now create an `ansible.cfg` file in the current directory if one does not already exist:

Copy

```
$ nano ansible.cfg
```

For now, add a `[defaults]` section and point Ansible to the inventory you just created:

ansible.cfg

```
[defaults]
inventory = ./hosts
```

When you are ready, continue on to the different ways of supplying a password.

Using an interactive prompt

The most straightforward way to decrypt content at runtime is to have Ansible prompt you for the password. Add the `--ask-vault-pass` flag to any `ansible` or `ansible-playbook` command, and Ansible will prompt you for a password to use when it encounters vault-protected content.

For example, to copy the contents of a vault-encrypted file to a host, you could use the `copy` module with the `--ask-vault-pass` flag. If the file contains real sensitive data, you most likely want to lock down access on the remote host with permission and ownership restrictions.

> **Note:** This example uses `localhost` as the target host to minimize the number of servers required, but the results are the same as if the host were truly remote.

Copy

```
$ ansible --ask-vault-pass -bK -m copy -a 'src= secret_key dest=/tmp/ secre
```

The task specifies that the file's ownership should change to `root`, so administrative privileges are required. The `-bK` flag tells Ansible to prompt for the `sudo` (become) password on the target host, so you are asked for that first, then for the Vault password:

Output Copy

```
BECOME password:
Vault password:
```

When the passwords are provided, Ansible executes the task, using the Vault password for any encrypted files it finds. Keep in mind that all files referenced during a single execution must use the same password:

Output Copy

```
localhost | SUCCESS => {
    "changed": true,
    "checksum": "7a2eb5528c44877da9b0250710cba321bc6dac2d",
    "dest": "/tmp/secret_key",
    "gid": 0,
    "group": "root",
    "md5sum": "270ac7da333dd1db7d5f7d8307bd6b41",
    "mode": "0600",
    "owner": "root",
    "size": 18,
    "src": "/home/sammy/.ansible/tmp/ansible-tmp-1480978964.81-19664560697290
    "state": "file",
    "uid": 0
}
```

Prompting for a password is secure, but it can be tedious on repeated runs and it hinders automation, so the next sections cover alternatives.

Using a vault password file

If you do not want to type the Vault password each time, you can store it in a file and reference that file during execution. For example, put your password in a `.vault_pass` file:

Copy

```
$ echo ' my_vault_password ' > .vault_pass
```

Note: A vault password file is as sensitive as the password itself. Restrict its permissions so only your user can read it, and never commit it to version control.

Lock down the file so that only your user can read it, then, if you use version control, add it to your ignore file so you do not commit it by accident:

Copy

```
$ chmod 600 .vault_pass
$ echo '.vault_pass' >> .gitignore
```

Now reference the file with the `--vault-password-file` flag. You can complete the same task from the last section without an interactive Vault prompt:

Copy

```
$ ansible --vault-password-file=.vault_pass -bK -m copy -a 'src= secret_key
```

You are not prompted for the Vault password this time:

Output

Copy

```
localhost | SUCCESS => {
    "changed": false,
    "checksum": "52d7a243aea83e6b0e478db55a2554a8530358b0",
    "dest": "/tmp/secret_key",
    "gid": 0,
    "group": "root",
    "mode": "0600",
    "owner": "root",
    "path": "/tmp/secret_key",
    "size": 8,
    "state": "file",
    "uid": 0
}
```

Reading the password file automatically

To avoid providing a flag at all, set the `ANSIBLE_VAULT_PASSWORD_FILE` environment variable with the path to the password file:

Copy

```
$ export ANSIBLE_VAULT_PASSWORD_FILE=./.vault_pass
```

You can now run the command without the `--vault-password-file` flag for the current session:

Copy

```
$ ansible -bK -m copy -a 'src= secret_key  dest=/tmp/ secret_key  mode=0600 ow
```

To make Ansible aware of the password file location across sessions, edit your `ansible.cfg` file. Open the local `ansible.cfg` you created earlier:

Copy

```
$ nano ansible.cfg
```

In the `[defaults]` section, set the `vault_password_file` option to the location of your password file. This can be a relative or absolute path, whichever is most useful for you:

ansible.cfg

```
[defaults]
. . .
vault_password_file = ./.vault_pass
```

Now, when you run commands that require decryption, you are no longer prompted for the vault password. As a bonus, `ansible-vault` also uses the password in the file automatically when creating new files with `ansible-vault create` or encrypting existing ones with `ansible-vault encrypt` .

Reading the password from an environment variable

You may worry about accidentally committing your password file to your repository. While Ansible has an environment variable that points to the *location* of a password file, it does not have one for setting the password value itself. However, if your password file is executable, Ansible runs it as a script and uses the resulting output as the password. The following short script reads the password from an environment variable instead of storing it on disk.

Open your `.vault_pass` file in your editor:

Copy

```
$ nano .vault_pass
```

Replace the contents with the following script. Note the parentheses on `print()`, which are required in Python 3:

.vault_pass Copy

```
#!/usr/bin/env python3

import os
print(os.environ['VAULT_PASSWORD'])
```

Make the file executable:

Copy

```
$ chmod +x .vault_pass
```

You can then set and export the `VAULT_PASSWORD` environment variable, which is available for your current session:

Copy

```
$ export VAULT_PASSWORD= my_vault_password
```

You have to do this at the beginning of each Ansible session, which may sound inconvenient. However, it effectively guards against accidentally committing your Vault password, since the password itself is never written to disk. This pattern is also the foundation of the CI/CD approach described later.

Working with multiple vault passwords

As projects grow, a single password rarely fits every secret. You often want separate passwords for development, staging, and production, so that a developer with the dev password cannot decrypt production secrets. Ansible handles this with *vault IDs*, which attach a label to each password.

A vault ID uses the syntax `label@source`, where `source` is either a password file or the keyword `prompt`. For example, to encrypt a string with a production vault ID and be prompted for the password, run the following:

Copy

```
$ ansible-vault encrypt_string --vault-id  prod@prompt  ' RealProductionPassw
```

When you use a labeled vault ID, the label is recorded in the encrypted block's header, which changes from version `1.1` to `1.2` and includes the label:

Output Copy

```
$ANSIBLE_VAULT;1.2;AES256; prod
```

When you run a playbook, you can supply several vault IDs at once, and Ansible tries each one as needed. The following passes a development password from a file and prompts for the production password:

Copy

```
$ ansible-playbook site.yml --vault-id  dev@dev_pass  --vault-id  prod@prompt
```

When decrypting, Ansible tries the password whose label matches the encrypted content first, then falls back to trying the others in the order you provided them. Vault IDs are most useful when you separate secrets by environment, when different team members hold different passwords, or when you rotate one environment's password without touching the others.

Using vault-encrypted variables with regular variables

While Ansible Vault can encrypt arbitrary files, it is most frequently used to protect sensitive variables. This section works through an example that transforms a regular variables file into a configuration that balances security and usability. It complements the inline `encrypt_string` approach shown earlier by splitting secrets into a separate encrypted file.

Setting up the example

For this example, pretend you are configuring a database server, without actually installing a database. When you created the `hosts` file earlier, you placed the `localhost` entry in a group called `database` to prepare for this step. Databases usually require a mixture of sensitive and nonsensitive variables, which you can assign in a `group_vars` directory in a file named after the group:

Copy

```
$ mkdir -p group_vars
$ nano group_vars/database.yml
```

Inside the `group_vars/database.yml` file, add some typical variables. Some, like the MySQL port number, are not secret and can be freely shared, while others, like the database password, are confidential:

group_vars/database.yml

Copy

```
---
# nonsensitive data
mysql_port: 3306
mysql_host: 10.0.0.3
mysql_user: fred

# sensitive data
mysql_password:  supersecretpassword
```

You can test that all of the variables are available to your host with Ansible's `debug` module and the `hostvars` variable:

Copy

```
$ ansible -m debug -a 'var=hostvars[inventory_hostname]'  database
```

The output confirms that every variable you defined is applied to the host:

Output Copy

```
localhost | SUCCESS => {
    "hostvars[inventory_hostname]": {
        "ansible_check_mode": false,
        "ansible_version": {
            "full": "2.18.1",
            "major": 2,
            "minor": 18,
            "revision": 1,
            "string": "2.18.1"
        },
        "group_names": [
            "database"
        ],
        "groups": {
            "all": [
                "localhost"
            ],
            "database": [
                "localhost"
            ],
            "ungrouped": []
        },
        "inventory_dir": "/home/sammy",
        "inventory_file": "hosts",
```

```
        "inventory_hostname": "localhost",
        "inventory_hostname_short": "localhost",
         "mysql_host": "10.0.0.3",
         "mysql_password": "supersecretpassword",
         "mysql_port": 3306,
         "mysql_user": "fred",
        "omit": "__omit_place_holder__1c934a5a224ca1d235ff05eb9bda22044a6fb40
        "playbook_dir": "."
    }
}
```

The output confirms that all of the variables are applied to the host. However, the `group_vars/database.yml` file currently holds all of the variables together. This means you can either leave it unencrypted, which is a security concern because of the database password, or encrypt all of the variables, which creates usability and collaboration problems.

Moving sensitive variables into Ansible Vault

To solve this, make a distinction between sensitive and nonsensitive variables so you can encrypt confidential values while still easily sharing the rest. To do so, split the variables between two files.

You can use a variable *directory* in place of a variable *file* to apply variables from more than one file. First, rename the existing file from `database.yml` to `vars.yml`, which will be your unencrypted variable file:

Copy

```
$ mv group_vars/database.yml group_vars/vars.yml
```

Next, create a directory with the same name as the old variable file, and move the `vars.yml` file inside it:

Copy

```
$ mkdir group_vars/database
$ mv group_vars/vars.yml group_vars/database/
```

You now have a variable directory for the `database` group instead of a single file, plus one unencrypted variable file. Since you will be encrypting your sensitive variables, remove them from the unencrypted file. Edit `group_vars/database/vars.yml` to remove the confidential data:

Copy

```
$ nano group_vars/database/vars.yml
```

In this case, remove the `mysql_password` variable so the file looks like this:

group_vars/database/vars.yml Copy

```
---
# nonsensitive data
mysql_port: 3306
mysql_host: 10.0.0.3
mysql_user: fred
```

Next, create a vault-encrypted file in the directory that will live alongside the unencrypted `vars.yml` file:

Copy

```
$ ansible-vault create group_vars/database/vault.yml
```

In this file, define the sensitive variables that used to be in the `vars.yml` file. Use the same variable names, but prepend the string `vault_` to indicate that these variables are defined in the vault-protected file:

group_vars/database/vault.yml Copy

```
---
vault_mysql_password: supersecretpassword
```

Save and close the file. The resulting directory structure looks like this:

Copy

```
.
├── . . .
├── group_vars/
│   └── database/
│       ├── vars.yml
│       └── vault.yml
└── . . .
```

At this point, the variables are separate and only the confidential data is encrypted. This is secure, but it has reduced usability: while the goal was to protect sensitive *values*, you have also unintentionally hidden the variable *names*. It is not clear which variables are assigned without referencing more than one file, and you probably still want to share the variable names even while restricting access to the values. To address this, the Ansible project recommends a slightly different approach.

Referencing vault variables from unencrypted variables

When you moved the sensitive data to the vault-protected file, you prefaced the variable names with `vault_`, so `mysql_password` became `vault_mysql_password`. You can add the original variable names back to the unencrypted file and, instead of setting them to sensitive values directly, use Jinja2 templating to reference the encrypted variables. This way, you can see all of the defined variables in a single file, while the confidential values remain in the encrypted file.

Open the unencrypted variables file again:

Copy

```
$ nano group_vars/database/vars.yml
```

Add the `mysql_password` variable again, this time using Jinja2 templating to reference the variable defined in the vault-protected file:

group_vars/database/vars.yml Copy

```
---
# nonsensitive data
```

```
mysql_port: 3306
mysql_host: 10.0.0.3
mysql_user: fred

# sensitive data
mysql_password:  "{{ vault_mysql_password }}"
```

The `mysql_password` variable is set to the value of `vault_mysql_password`, which is defined in the vault file. With this method, you can understand all of the variables that apply to hosts in the `database` group by viewing the `group_vars/database/vars.yml` file, while the sensitive parts stay obscured behind the Jinja2 reference. You only need to open `group_vars/database/vault.yml` when the values themselves need to be viewed or changed.

You can check that all of the `mysql_*` variables are still correctly applied using the same method as before.

> **Note:** If your Vault password is not being applied automatically through a password file, add the `--ask-vault-pass` flag to the command below.

Copy

```
$ ansible -m debug -a 'var=hostvars[inventory_hostname]'  database
```

The output shows both variables resolving to the same secret value:

Output Copy

```
localhost | SUCCESS => {
    "hostvars[inventory_hostname]": {
        "ansible_check_mode": false,
        "ansible_version": {
            "full": "2.18.1",
            "major": 2,
            "minor": 18,
            "revision": 1,
            "string": "2.18.1"
        },
        "group_names": [
            "database"
```

```
        ],
        "groups": {
            "all": [
                "localhost"
            ],
            "database": [
                "localhost"
            ],
            "ungrouped": []
        },
        "inventory_dir": "/home/sammy/vault",
        "inventory_file": "./hosts",
        "inventory_hostname": "localhost",
        "inventory_hostname_short": "localhost",
         "mysql_host": "10.0.0.3",
         "mysql_password": "supersecretpassword",
         "mysql_port": 3306,
         "mysql_user": "fred",
        "omit": "__omit_place_holder__6dd15dda7eddafe98b6226226c7298934f666fc
        "playbook_dir": ".",
         "vault_mysql_password": "supersecretpassword"
    }
}
```


Both `vault_mysql_password` and `mysql_password` are accessible, and this duplication is harmless and will not affect your use of the system.

Ansible Vault compared with environment variables and external secrets managers

Ansible Vault is not the only way to handle secrets in automation, and choosing the right tool depends on your team size, your workflow, and how your secrets are consumed. The following table compares the three most common approaches.

Method	Encryption at rest	Safe to commit to version control	CI/CD friendly	Complexity
Ansible Vault	Yes (AES-256)	Yes (encrypted files only)	Yes, with a password injected at runtime	Low

Method	Encryption at rest	Safe to commit to version control	CI/CD friendly	Complexity
Plain environment variables	No	No (never commit secrets)	Yes, via pipeline secrets	Very low
External secrets manager	Yes, with rotation and auditing	Yes (no secret stored in repo)	Yes, via API or plugin	High

Ansible Vault is the best fit when you want secrets encrypted at rest and stored alongside your playbooks in the same repository, which suits small to mid-sized teams and infrastructure-as-code workflows. Plain environment variables are simplest for a single secret consumed at runtime, but they offer no encryption at rest and must never be committed. An external secrets manager is the strongest option for large organizations that need centralized rotation, fine-grained access control, and audit logging, at the cost of additional infrastructure to run and integrate. Many teams combine approaches, for example using Ansible Vault for project-level secrets and an external manager for organization-wide credentials.

Best practices for managing secrets with Ansible Vault

A few habits keep a vault-based workflow both secure and maintainable:

- Keep the vault password file outside version control and restrict it with `chmod 600`, and add unencrypted secret files to `.gitignore` so they are never committed.
- Prefer `ansible-vault encrypt_string` for individual values in shared repositories so reviewers can still see variable names, and reserve whole-file encryption for files that are entirely sensitive.
- Use a clear naming convention, such as the `vault_` prefix, so it is obvious which variables come from an encrypted source.
- Separate vault IDs by environment (for example `dev`, `staging`, and `prod`) so that access to one environment's secrets does not grant access to another's.

- Rotate vault passwords on a defined schedule using `ansible-vault rekey`, and rotate immediately if a password may have been exposed.
- Store the vault passwords themselves in a dedicated password manager or secrets manager, because losing a vault password means the encrypted data cannot be recovered.
- Use `no_log: true` on any task that handles a decrypted secret to prevent the value from appearing in task output, logs, or Ansible Tower / AWX job results. For example:

  Copy

  ```
  - name: Create database user
  community.mysql.mysql_user:
      name: "{{ mysql_user }}"
      password: "{{ mysql_password }}"
      state: present
  no_log: true
  ```

  Vault protects data on disk; `no_log` protects it at runtime. Both controls together give you defense in depth.
- Although `ansible-vault encrypt` works on any file including entire playbooks, whole-playbook encryption is rarely practical in team settings because it prevents anyone from reading the task list without decrypting it first. Prefer encrypting only the variables files that contain secrets, and keep your playbooks in plain text.

FAQs

1. Can you use Ansible Vault to store sensitive information?

Yes, Ansible Vault is designed specifically to store sensitive information such as passwords, API keys, private keys, and TLS certificates by encrypting them at rest. It is appropriate for any secret you need available during a playbook run but do not want visible in plain text in your repository. Keep in mind what it does not protect: once Ansible decrypts a value at runtime, that value exists in memory as a normal variable and can appear in verbose output or logs, so it secures data on disk rather than during execution.

2. How do you encrypt a file with Ansible Vault?

Run `ansible-vault encrypt filename.yml` to encrypt an existing file, or `ansible-vault create filename.yml` to create and encrypt a new one. In both cases Ansible prompts you to enter and confirm a vault password, then writes the encrypted content back to disk in place of the original. You can confirm the result with `cat filename.yml`, which shows the `$ANSIBLE_VAULT` header followed by the encrypted payload instead of your plain text.

3. How secure is Ansible Vault?

Ansible Vault uses AES-256 encryption in CTR mode and derives its key from your password with PBKDF2 using HMAC-SHA256 over 10,000 iterations, which are strong, industry-standard algorithms. Because the key is derived directly from your password, the real-world security depends on choosing a long, unique password and storing it safely rather than on the algorithm. Vault protects data at rest only, so it does not prevent a decrypted secret from being exposed in memory, logs, or verbose output during a run.

4. Which feature should you use to securely manage sensitive data such as passwords in your Ansible playbook?

Ansible Vault is the built-in feature for securely managing sensitive data such as passwords in your playbooks. It encrypts secrets at rest and integrates directly with the `ansible` and `ansible-playbook` commands, which decrypt the content automatically at runtime when you supply the password. For organization-wide credential management with rotation and auditing, you can complement Vault with an external secrets manager such as HashiCorp Vault.

5. What is the difference between `ansible-vault encrypt` and `ansible-vault encrypt_string`?

`ansible-vault encrypt` encrypts an entire file, so the whole file becomes an opaque encrypted block, while `ansible-vault encrypt_string` encrypts a single value and returns an inline YAML snippet you can paste into an otherwise plain-text variables file. Use whole-file encryption when a file contains only secrets, and use `encrypt_string` when you want most variables to stay readable and only specific values encrypted. The inline approach is generally friendlier for code review in shared repositories.

6. How do you use a vault password file instead of typing a password interactively?

Save the password in a file and reference it with the `--vault-password-file` flag, for example `ansible-playbook site.yml --vault-password-file .vault_pass`, or set the `ANSIBLE_VAULT_PASSWORD_FILE` environment variable to the file's path so you do not need the flag. You can also set `vault_password_file` in the `[defaults]` section of `ansible.cfg` to make it the default. In every case, restrict the file with `chmod 600` and add it to `.gitignore` so it is never readable by others or committed to version control.

7. Can Ansible Vault be used in CI/CD pipelines?

Yes, Ansible Vault works well in CI/CD pipelines. Store the vault password as a protected pipeline secret (for example, a masked CI/CD variable), then have the pipeline write it to a temporary file at the start of the run, pass that file with `--vault-password-file`, and delete the file when the run finishes. This keeps the password out of the repository and out of the build logs while still allowing fully automated, unattended playbook runs.

Here is an example GitHub Actions step:

Copy

```
- name: Run playbook
  env:
    VAULT_PASSWORD: ${{ secrets.VAULT_PASSWORD }}
  run: |
    echo "$VAULT_PASSWORD" > .vault_pass
    chmod 600 .vault_pass
    ansible-playbook site.yml --vault-password-file .vault_pass
    rm -f .vault_pass
```

For GitLab CI, the pattern is equivalent: set `VAULT_PASSWORD` as a masked CI/CD variable in your project settings and reference it the same way. Always delete the temporary file in the same step so it is removed even if the playbook fails.

8. What happens if you lose your Ansible Vault password?

If you lose a vault password, the data encrypted with it cannot be recovered, because the password is required to derive the decryption key and there is no backdoor or recovery mechanism. Your only options are to restore the original unencrypted data from another source or recreate the secrets. To avoid this situation, store vault passwords in a dedicated password manager or secrets manager, and consider rekeying important vaults on a schedule so the current password is always known and backed up.

Conclusion

In this guide, you learned how Ansible Vault encrypts data with AES-256. You also learned how to encrypt both whole files and individual variables, how to supply vault passwords interactively or through password files and environment variables, how to manage multiple passwords with vault IDs, and how Vault compares with other secrets-management approaches. With these tools, you can keep all of your configuration data in one place without compromising security.

To keep building your Ansible skills, explore these related DigitalOcean tutorials:

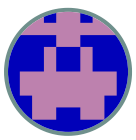
- How to Install and Configure Ansible on Ubuntu
- Configuration Management 101: Writing Ansible Playbooks
- How to Use Ansible: A Reference Guide
- How To Write Ansible Playbooks
- An Introduction to Configuration Management with Ansible

About the author(s)

Justin Ellingwood Author

See author profile

Former Senior Technical Writer at DigitalOcean, specializing in DevOps topics across multiple Linux distributions, including Ubuntu 18.04, 20.04, 22.04, as well as Debian 10 and 11.



Manikandan Kurup Editor

Senior Technical Content Engineer I

See author profile

With over 6 years of experience in tech publishing, Mani has edited and published more than 75 books covering a wide range of data science topics. Known for his strong attention to detail and technical knowledge, Mani specializes in creating clear, concise, and easy-to-understand content tailored for developers.

Category: Tutorial

Tags: Security Ubuntu Ansible Configuration Management

Still looking for an answer?

Ask a question

Search for more help

Was this helpful?

Yes

No

Comments(4) Follow-up questions(0)

Leave a comment...

This textbox defaults to using Markdown to format your answer.

You can type !ref in this text area to quickly search our full set of tutorials, documentation & marketplace offerings and insert the link!

Sign in/up to comment

Patrick Artounian

July 1, 2017

Show less

Hey Justin, is there a best practice key length for the ansible vault pass? If so, what is it? Between 12-16 characters, etc?

Reply

(1) replies

Muhammad Hannan

November 20, 2017

Show less

Hey, This has been very good and interesting article. I am recently looking for similar kind of solution that can hold my Database, API credentials encrypted (instead of I write them directly in my application code). Will this solution help me? I have posted my question here in detail: https://www.digitalocean.com/community/questions/how-to-secure-important-credentials-used-in-node-js-application-code-in-separate-side-utility?answer=39512

Is it possible to use ansible-vault module instead of configuring whole Ansible platform? As it seems quite big system that I notice here: https://www.ansible.com/

What are your recommendations on it in reference to my specific need.

Reply

(1) replies

gitprodenv

January 9, 2020

This comment has been deleted

ahmed2021

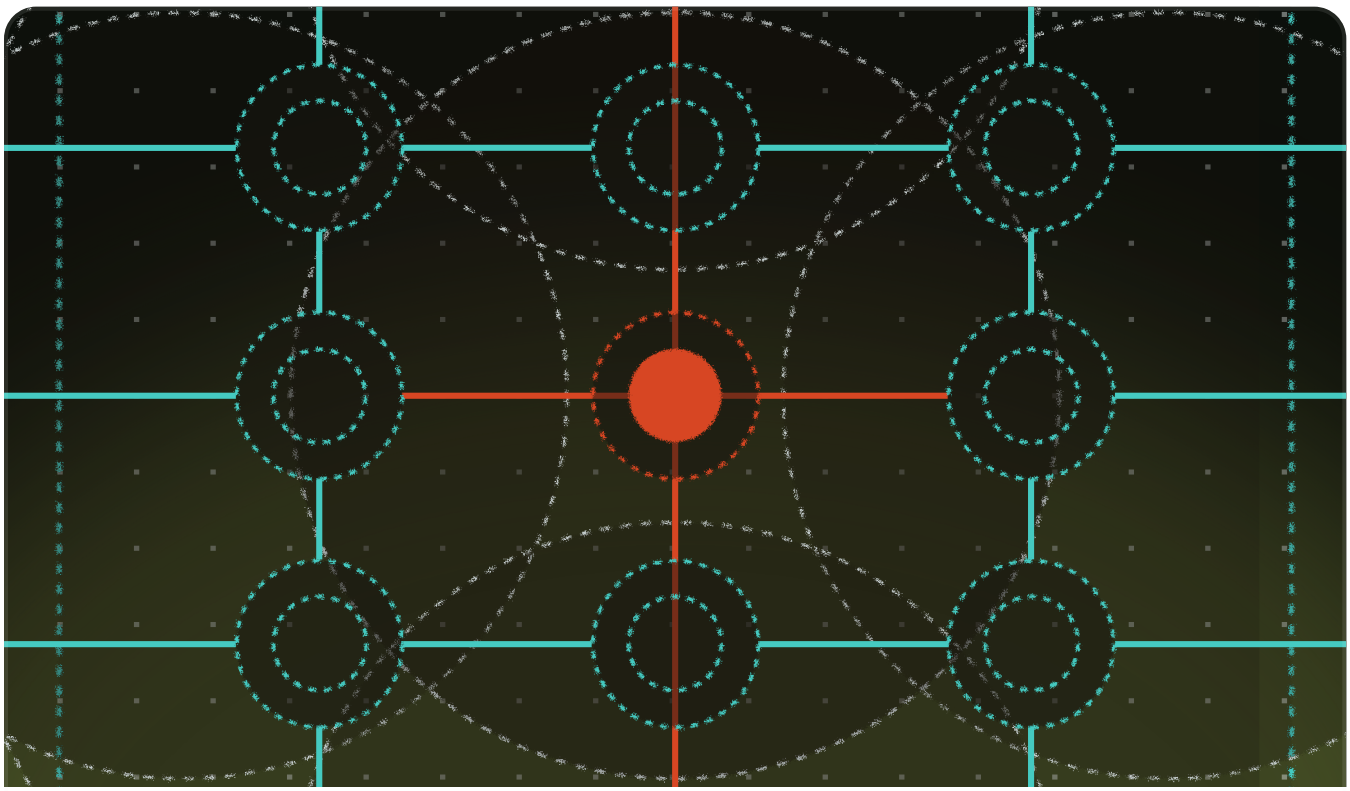
December 4, 2021

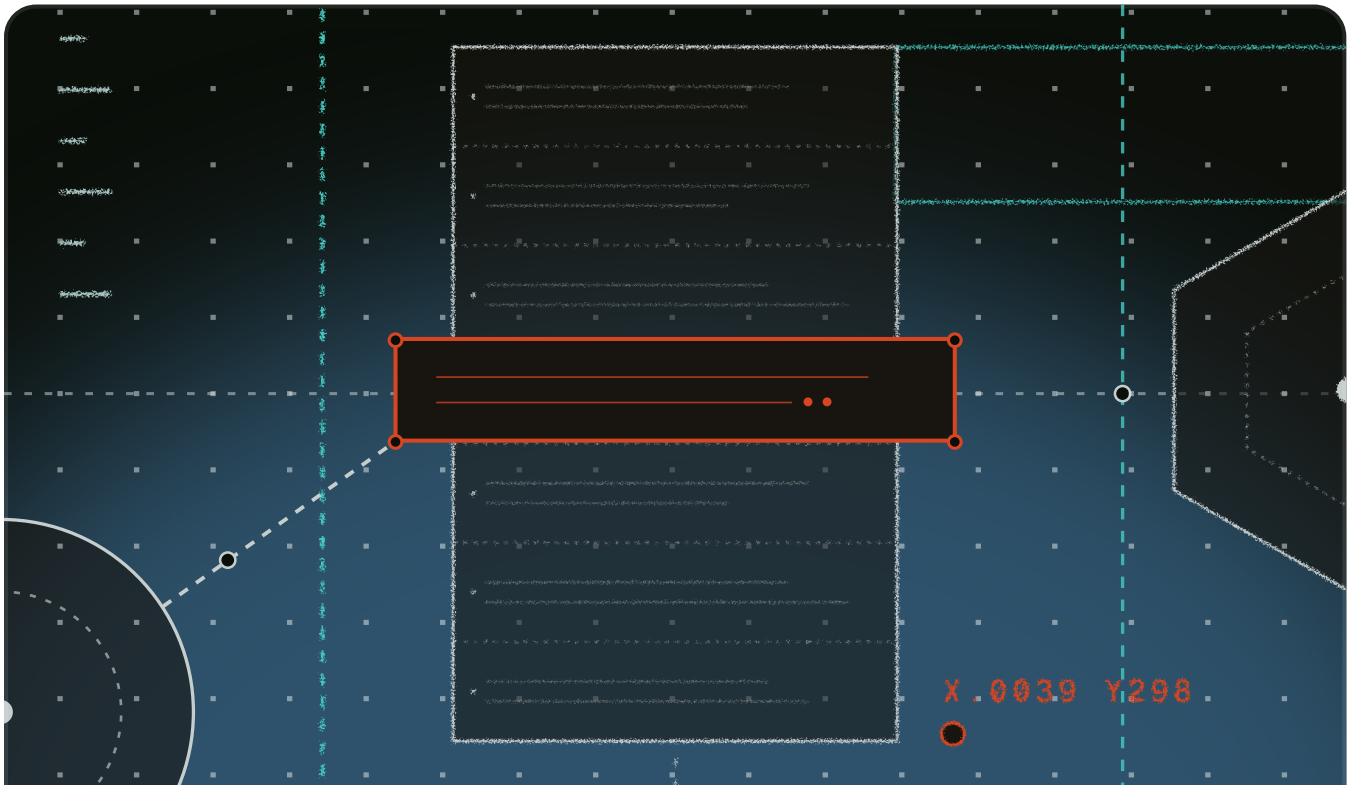
Show less

Hi Justin,

At the last part ‘Referencing Vault Variables from Unencrypted Variables’, can we add lots of dictionary of keys and value(passwords) in vault file and reference them in vars ?

Reply

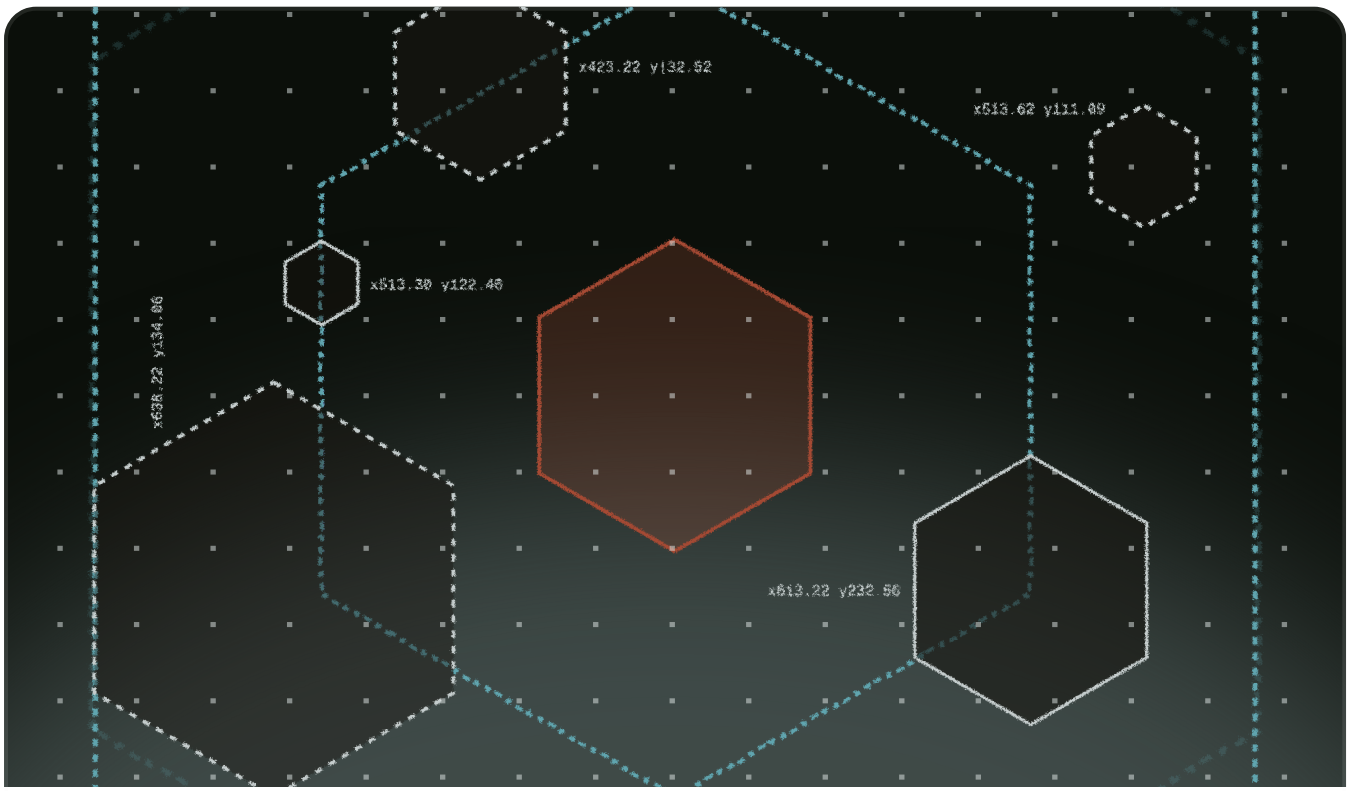




Документация DigitalOcean

Полная документация по каждому продукту DigitalOcean.

Узнать больше →



Ресурсы для стартапов и компаний, использующих искусственный интеллект

В The Wave есть все, что вам нужно знать о построении бизнеса: от привлечения финансирования до маркетинга вашего продукта.

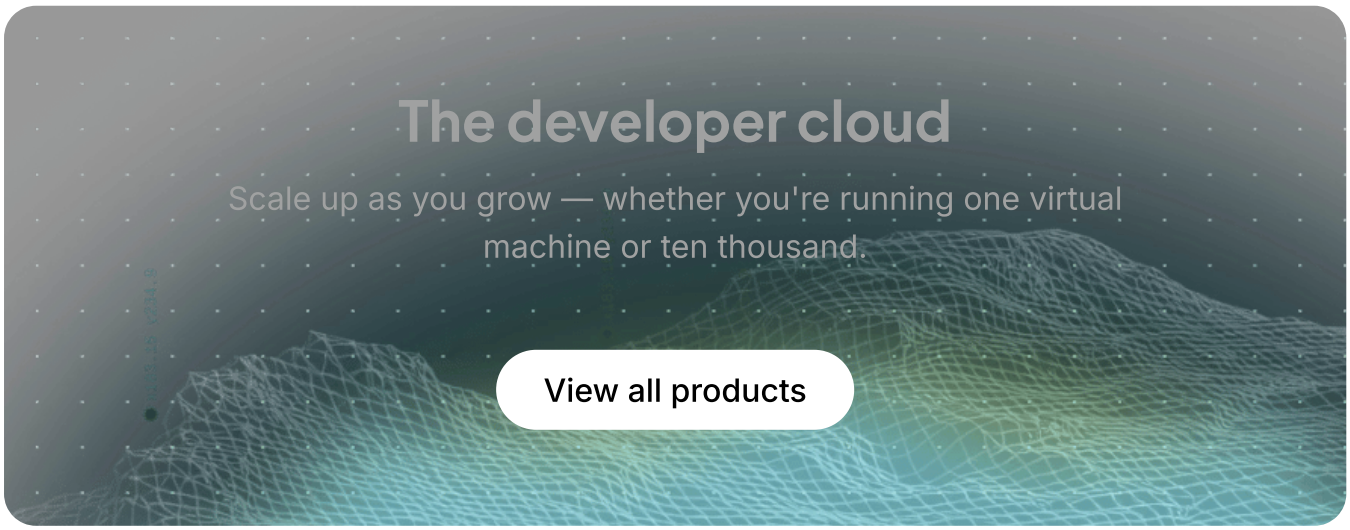
Узнать больше →

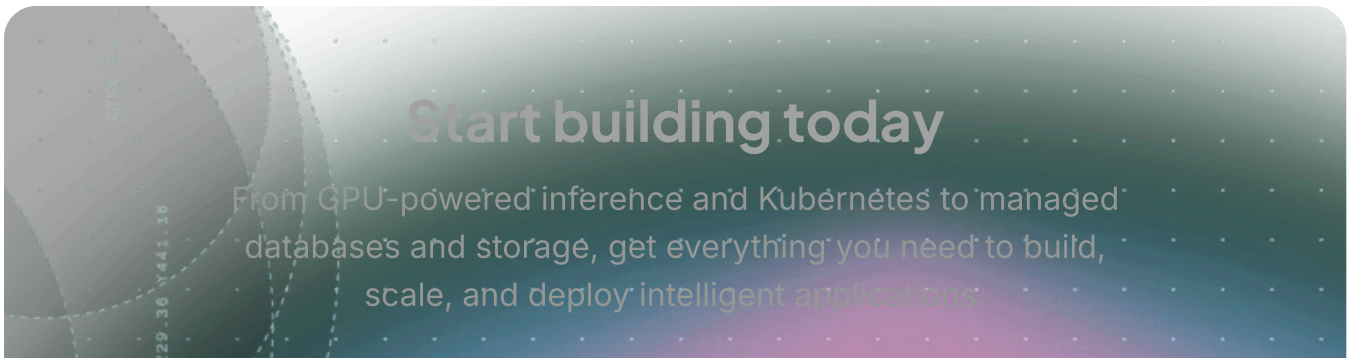
Будьте в курсе событий, подпишитесь на нашу рассылку

Электронная почта *

Эл. адрес

Submit





Sign up

Company

Products

Resources

Solutions

Contact